Jednokierunkowa sieć neuronowa typu MLP

Maciej Kusy

Katedra Podstaw Elektroniki

Spis treści

- Wstęp
- Pierwowzór sieci neuronowych – mózg
- Biologiczny model neuronu
- Model neuronu – transmisja sygnału
- Matematyczny model neuronu McCullocha–Pittsa
- Rodzaje sieci neuronowych
- Reguły uczenia nadzorowanego dla jednokierunkowych sieci neuronowych
- Algorytm wstecznej propagacji błędu
  - Aktywacje sigmoidalne
  - Aktywacje ReLU+softmax
- Inicjalizacja wag sieci
- Analiza procesu uczenia MLP
- Literatura

Wstęp

Sztuczna sieć neuronowa (SN) jest modelem obliczeniowym **inspirowanym budową i działaniem biologicznych sieci neuronowych**.

Przekształca dane wejściowe w sygnały wyjściowe.

SN składa się z połączonych elementów obliczeniowych zwanych **neuronami**. Każdy neuron przetwarza otrzymane sygnały i wyznacza sygnał wyjściowy (Zurada, 1992).

Połączeniom między neuronami przypisuje się **wagi**, które określają wpływ poszczególnych sygnałów wejściowych.

W sieci MLP uczenie polega na modyfikacji **wag i przesunięć** na podstawie danych uczących (Goodfellow et al., 2016, rozdz. 6).

Na działanie sieci wpływają jej **architektura**, funkcje aktywacji neuronów oraz sposób uczenia.

Architektura określa m.in. liczbę warstw, liczbę neuronów i sposób ich połączenia.

Pierwowzór sieci neuronowych – mózg

Mózg człowieka tworzy rozbudowana sieć współpracujących komórek.

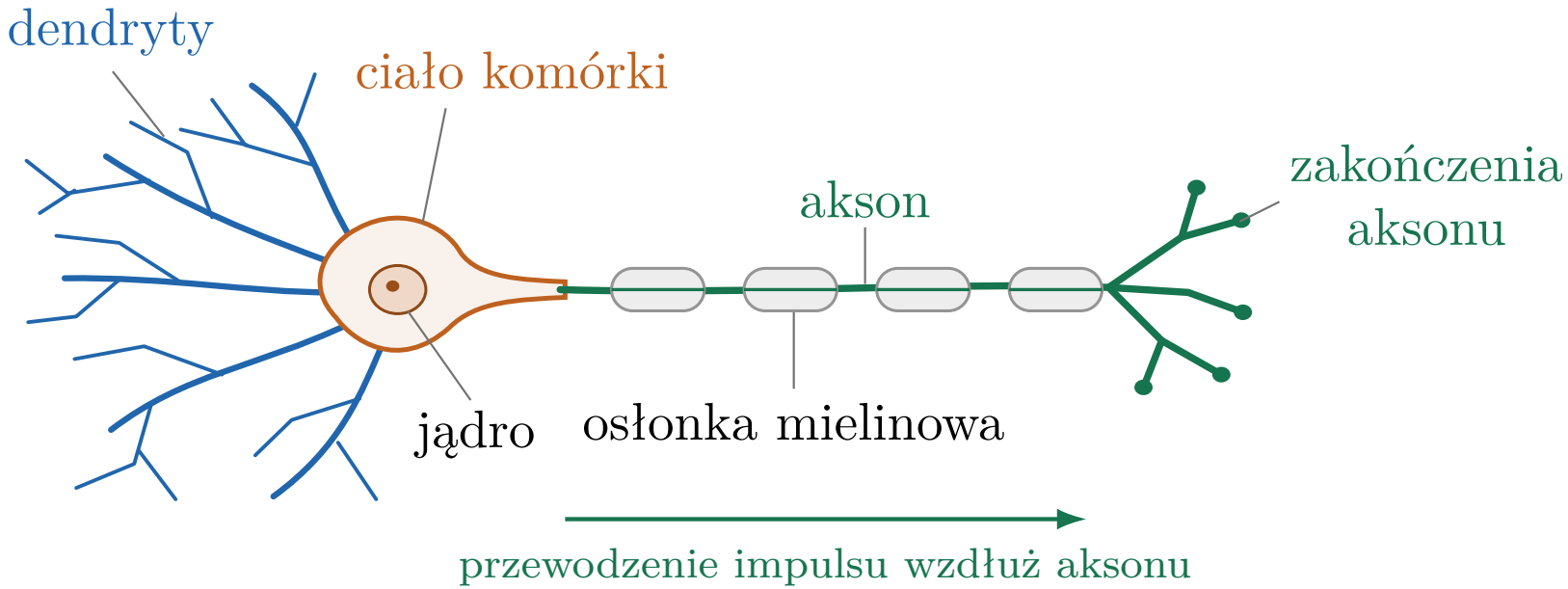
Szacunkowa liczba komórek w całym mózgu wynosi (Azevedo et al., 2009):

Rodzaj komórek	**Liczba**
neurony	ok. 86 mld
komórki nieneuronalne, w tym komórki glejowe	ok. 85 mld

Funkcje komórek (Purves et al., 2001):

- Neurony odbierają, przetwarzają i przekazują sygnały.
- Komórki glejowe m.in. wspierają funkcjonowanie neuronów i utrzymują odpowiednie warunki ich pracy.
- Neurony komunikują się za pośrednictwem **synaps**.
- Zmiany skuteczności połączeń synaptycznych są jednym z mechanizmów **uczenia i pamięci**.

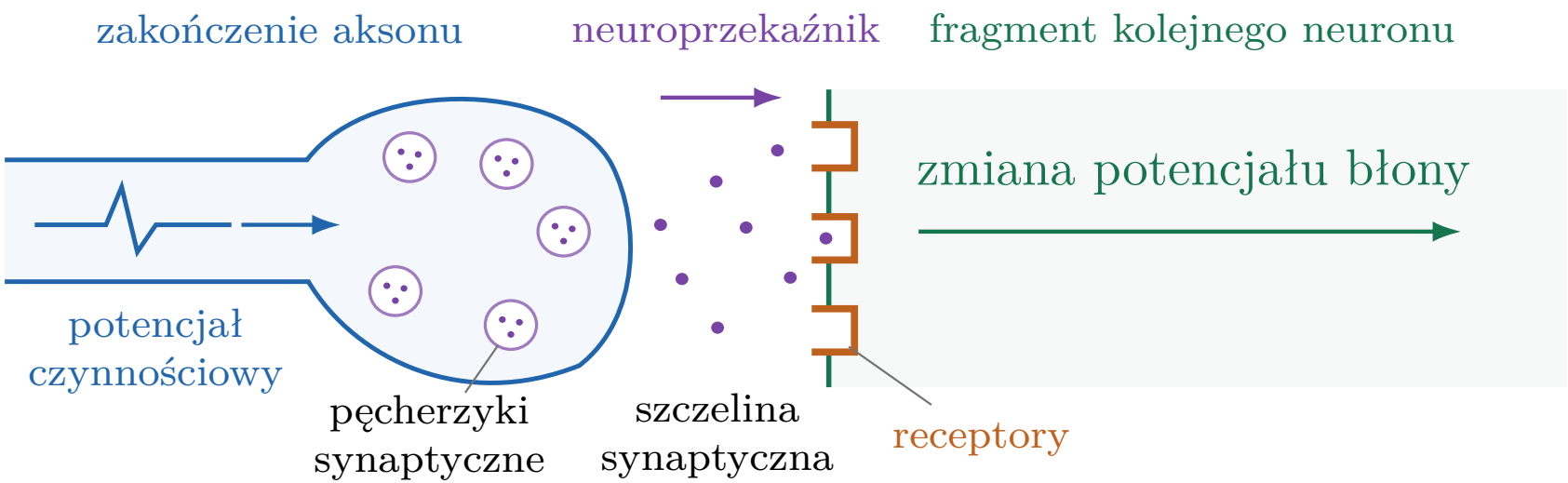
Budowa neuronu biologicznego (Purves et al., 2001)



- **Dendryty** i ciało komórki odbierają sygnały od innych neuronów.
- Integracja wielu sygnałów wpływa na powstanie **potencjału czynnościowego**.
- **Akson** przewodzi potencjały czynnościowe do zakończeń, które kontaktują się z kolejnymi komórkami.

Neuron biologiczny – przekazywanie sygnału

Na przykładzie **synapsy chemicznej** (Purves et al., 2001):



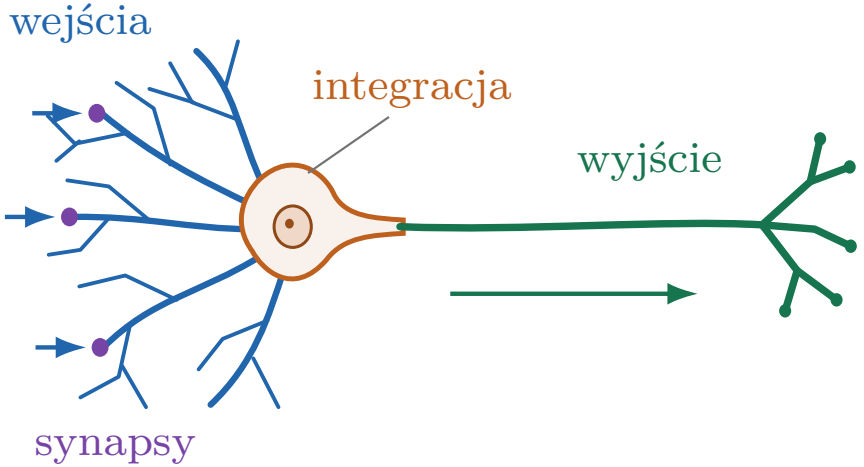
- Potencjał czynnościowy dociera do zakończenia aksonu.
- Uwolniony neuroprzekaźnik wiąże się z receptorami kolejnej komórki.
- Odpowiedź może pobudzać lub hamować neuron, zależnie od receptorów.

O powstaniu kolejnego impulsu decyduje **łączny wpływ wielu wejść**.

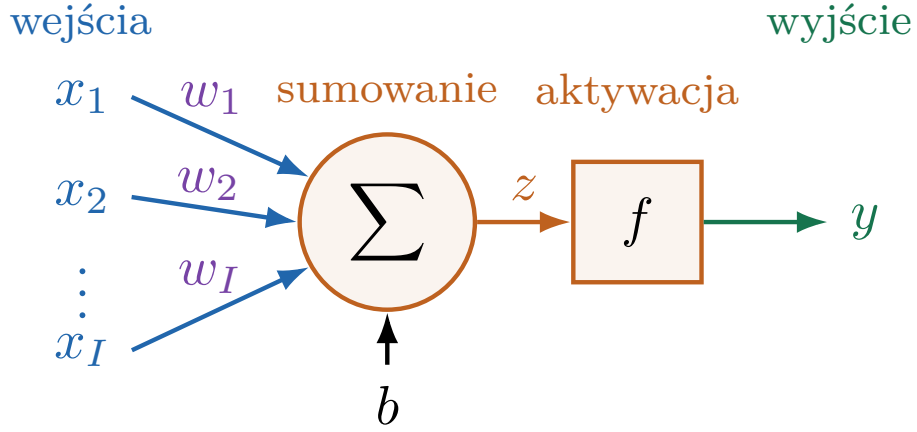
Matematyczny model neuronu

Neuron progowy – uogólnienie modelu (McCulloch and Pitts, 1943).

Neuron biologiczny



Model matematyczny



$$z = \sum_{i=1}^{I} w_i x_i + b, \qquad y = f(z) = \begin{cases} 1, & z \geqslant 0, \\ 0, & z < 0. \end{cases} \tag{1}$$

- x_i – sygnały wejściowe, w_i – wagi połączeń,
- b – przesunięcie (bias), z – pobudzenie neuronu,
- $y \in \{0, 1\}$ – sygnał wyjściowy.

Wybrane rodzaje sieci neuronowych

- **Sieci jednokierunkowe:** perceptron binarny, perceptrony wielowarstwowe (MLP), sieci radialne (RBF), probabilistyczne (PNN), typowe sieci splotowe (CNN).
- **Sieci rekurencyjne:** sieć Hopfielda, maszyna Boltzmanna, sieci do przetwarzania sekwencji, np. z komórkami LSTM.
- **Samoorganizujące się mapy (SOM)** – sieci Kohonena.
- **Sieci kontrpropagacyjne (CPN)** – łączące uczenie konkurencyjne z uczeniem nadzorowanym.
- **Inne:** sieci autoasocjacyjne, wielomianowe, o kaskadowej korelacji.

Określenie **sieć głęboka** dotyczy wielu kolejnych warstw przetwarzania. Głębokie mogą być m.in. sieci MLP i CNN[1].

MLP zwykle uczone są metodami gradientowymi z wykorzystaniem **propagacji wstecznej**.

[1] Kategorie nie tworzą rozłącznego podziału (Goodfellow et al., 2016).

Uczenie sieci neuronowych

Uczenie polega na dobieraniu **parametrów sieci** na podstawie danych.

W sieci MLP uczenie polega na modyfikacji wag połączeń oraz przesunięć.

W uczeniu nadzorowanym dla przykładów wejściowych znane są oczekiwane odpowiedzi.

Sieć uczy się na ich podstawie odwzorowania wejście-wyjście.

Istnieją trzy główne reguły uczenia (Zurada, 1992).

Dla pojedynczego neuronu:

1. reguła perceptronowa: neuron progowy,
2. reguła Widrowa-Hoffa: neuron liniowy,
3. reguła delta: neuron z różniczkowalną aktywacją.

Uczenie nadzorowane w SN

Zbiór uczący dla sieci z jednym wyjściem:

$$\mathcal{D} = \{(\mathbf{x}_l, t_l)\}_{l=1}^{L},\ \mathbf{x}_l \in \mathbb{R}^I, \tag{2}$$

gdzie:

- $\mathbf{x}_l$: wektor cech przykładu, t_l – oczekiwana odpowiedź;
- $y_l = F(\mathbf{x}_l; \boldsymbol{\theta})$: odpowiedź sieci, gdzie $\boldsymbol{\theta}$ obejmuje jej wagi i przesunięcia.
- Funkcja straty $\mathcal{L}(t_l, y_l)$ ocenia rozbieżność między odpowiedzią oczekiwaną i uzyskaną.

Przykładowy cel uczenia to minimalizacja średniej straty:

$$J(\boldsymbol{\theta}) = \frac{1}{L} \sum_{l=1}^{L} \mathcal{L}(t_l, y_l). \tag{3}$$

Funkcja straty dobierana jest do analizowanego zadania (kryterium: poprawne odpowiedzi dla nowych danych).

Uczenie pojedynczego neuronu – oznaczenia

Dla rozpatrywanej danej pary uczącej $(\mathbf{x}, t)$ pobudzenie z pojedynczego neuronu przyjmuje postać:

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^{I} w_i x_i + b, \qquad y = f(z), \tag{4}$$

gdzie:

- $\mathbf{w} = [w_1, \ldots, w_I]^T$ – wagi, b – przesunięcie;
- t – oczekiwana odpowiedź;
- f – funkcja aktywacji, y – odpowiedź neuronu.

W kroku k aktualizujemy wagi oraz przesunięcie:

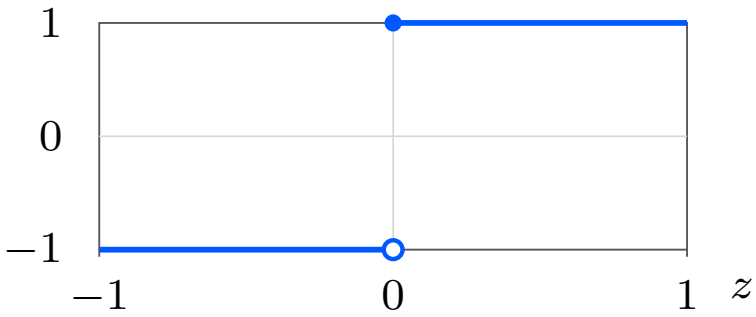
$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta\, \mathbf{e}_w^{(k)}, \qquad b^{(k+1)} = b^{(k)} + \eta\, e_b^{(k)}, \qquad \eta > 0.$$

Sygnały korekcyjne $\mathbf{e}_w^{(k)}$ oraz $e_b^{(k)}$ wyznaczane są na podstawie błędu odpowiedzi neuronu, zgodnie z przyjętą regułą uczenia.

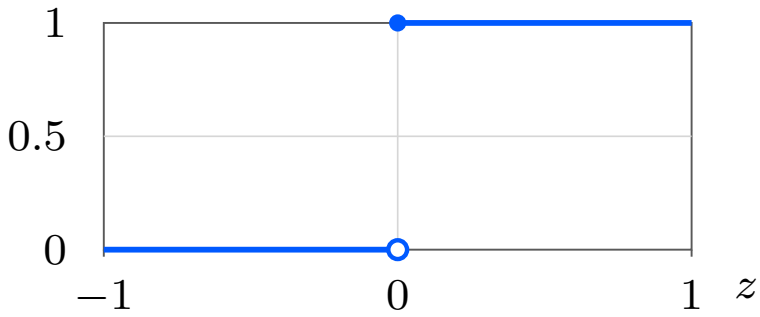
Funkcja f może być m.in. progowa, liniowa lub sigmoidalna.

Przykłady funkcji aktywacji

Progowa bipolarna

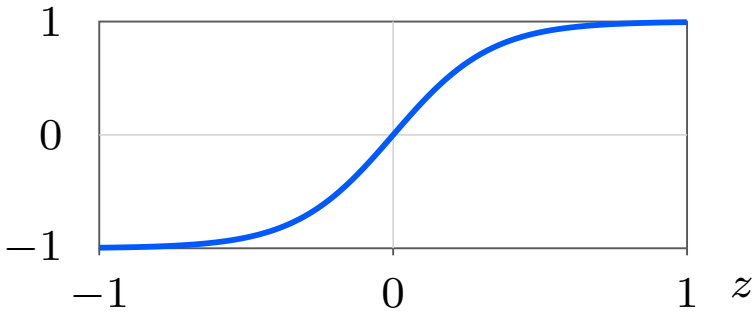


Progowa unipolarna



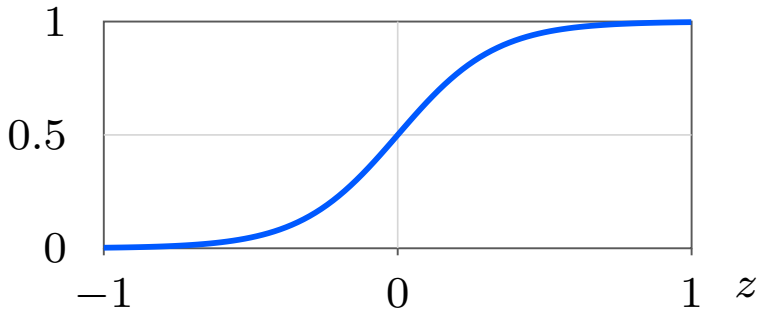
Sigmoidalna bipolarna

$f(z) = \frac{2}{1+e^{-\alpha z}} - 1$



Sigmoidalna unipolarna

$f(z) = \frac{1}{1+e^{-\alpha z}}$



Argument: $z = \mathbf{w}^T\mathbf{x} + b$. Dla f. progowych: $f(0) = 1$. Dla f. sigmoidalnych: $\alpha = 6$.

Reguła perceptronowa (Rosenblatt, 1958)

Neuron progowy przypisuje rekord do jednej z dwóch klas.

Dla kodowania unipolarnego $t, y \in \{0, 1\}$:

$$y = f(\mathbf{w}^T \mathbf{x} + b) = f(z) = \begin{cases} 1, & z \geqslant 0, \\ 0, & z < 0. \end{cases}$$

Reguła aktualizacji wykorzystuje różnicę $t - y$:

$$\Delta \mathbf{w} = \eta(t - y)\mathbf{x}, \qquad \Delta b = \eta(t - y). \tag{5}$$

$$\Delta w_i = \eta(t - y)x_i. \tag{6}$$

- Aktualizacja następuje tylko wtedy, gdy $y \neq t$.
- Dla danych liniowo separowalnych i stałego $\eta > 0$ reguła znajduje rozwiązanie poprawnie klasyfikujące zbiór uczący.

Inicjalizacja: np. $\mathbf{w} = \mathbf{0}$ i $b = 0$. Reguła działa też przy kodowaniu bipolarnym (Zurada, 1992).

Reguła Widrowa-Hoffa (Widrow, 1962)

Neuron ma **liniową funkcję aktywacji**: $y = z = \mathbf{w}^T\mathbf{x} + b$.

Dla oczekiwanej odpowiedzi t definiujemy **błąd kwadratowy**:

$$E = \frac{1}{2}(t - z)^2. \tag{7}$$

Celem uczenia jest zmniejszanie E. Zmiany parametrów wynoszą:

$$\Delta\mathbf{w} = \eta(t - z)\mathbf{x}, \qquad \Delta b = \eta(t - z). \tag{8}$$

Dla pojedynczej wagi:

$$\Delta w_i = \eta(t - z)x_i. \tag{9}$$

Przykład porównawczy: $t = 1$ oraz $z = 0.6$.

- **Reguła perceptronowa**: neuron zwraca **1** ($z \geqslant 0$).
  Odpowiedź jest poprawna: parametry się nie zmieniają.
- **Reguła Widrowa-Hoffa**: neuron zwraca **0.6** ($t - z = 0.4 \neq 0$).
  Ma miejsce korekta parametrów.

Reguła delta: wyznaczanie gradientu

Założenie: dla $y = f(z)$, funckja f ma pochodną w rozpatrywanym punkcie.

Monotoniczność nie jest potrzebna do wyprowadzenia aktualizacji.

$$E = \frac{1}{2}(t - y)^2 = \frac{1}{2}\left[t - f(\mathbf{w}^T\mathbf{x} + b)\right]^2. \tag{10}$$

Wagi i przesunięcie wpływają na z, następnie na y, a w rezultacie na E.

Na podstawie **reguły łańcuchowej** (Goodfellow et al., 2016):

$$\frac{\partial E}{\partial w_i} = -(t - y)f'(z)x_i. \tag{11}$$

W postaci wektorowej:

$$\nabla_{\mathbf{w}} E = -(t - y)f'(z)\mathbf{x}, \qquad \frac{\partial E}{\partial b} = -(t - y)f'(z). \tag{12}$$

$f'(z)$: lokalna wrażliwość wyjścia neuronu na zmianę jego pobudzenia.

Reguła delta: aktualizacja parametrów

Parametry zmieniają się w kierunku przeciwnym do gradientu.

Wielkość kroku określa $\eta > 0$:

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} E = \eta (t - y) f'(z) \mathbf{x}. \tag{13}$$

Zatem:

$$\Delta w_i = \eta (t - y) f'(z) x_i, \qquad \Delta b = \eta (t - y) f'(z). \tag{14}$$

Przy niezerowym gradiencie **dostatecznie mały krok** zmniejsza stratę dla rozpatrywanego przykładu. Zbyt duży krok może ją zwiększyć.

! $f(z) = z \rightarrow f'(z) = 1$: **reguła Widrowa-Hoffa**.

☎ W MLP neurony ukryte nie mają bezpośrednio zadanych odpowiedzi t.

Sygnały błędu potrzebne do obliczenia ich gradientów wyznacza **propagacja wsteczna** (Rumelhart, 1986).

Uczenie pojedynczego neuronu: podsumowanie

Aktualizacje dla pary $(\mathbf{x}, t)$, gdzie $z = \mathbf{w}^T\mathbf{x} + b$ i $y = f(z)$ (Zurada, 1992):

Reguła	**Aktywacja**	$\boldsymbol{\Delta w_i}$	$\boldsymbol{\Delta b}$
Perceptronowa	progowa	$\eta(t-y)x_i$	$\eta(t-y)$
Widrowa–Hoffa	liniowa	$\eta(t-z)x_i$	$\eta(t-z)$
Delta	z pochodną	$\eta(t-y)f'(z)x_i$	$\eta(t-y)f'(z)$

Reguła Widrowa-Hoffa jest **liniowym przypadkiem** reguły gradientowej.

Co wynika z tabeli?

W każdej regule zmiana wagi w_i zawiera czynnik x_i. Jeśli dla danego przykładu $x_i = 0$, to $\Delta w_i = 0$.

Przesunięcie b można traktować jak wagę przy stałym wejściu $x_0 = 1$.

Wsteczna propagacja błędu

Wsteczna propagacja pozwala wyznaczyć, jak parametry sieci wpływają na błąd jej odpowiedzi.

Podstawy tej metody opisano w latach 70. (Werbos, 1974); w uczeniu sieci wielowarstwowych upowszechniła ją praca (Rumelhart, 1986).

Uczenie MLP obejmuje trzy etapy:

1. **Przejście w przód:** od wejść x_i obliczamy odpowiedzi neuronów ukrytych z_j, a następnie odpowiedzi neuronów wyjściowych y_k.
2. **Przejście wstecz:** porównujemy y_k z oczekiwanymi wartościami t_k i wyznaczamy wskaźniki korekty δ_k oraz δ_j.
3. **Aktualizacja parametrów:** zmieniamy wagi $w_{ji}^{(1)}, w_{kj}^{(2)}$ i przesunięcia $b_j^{(1)}, b_k^{(2)}$.

Wskaźniki δ_j pozwalają korygować również wagi neuronów ukrytych, dla których nie podajemy bezpośrednio oczekiwanych odpowiedzi.

Uczenie MLP z jedną warstwą ukrytą

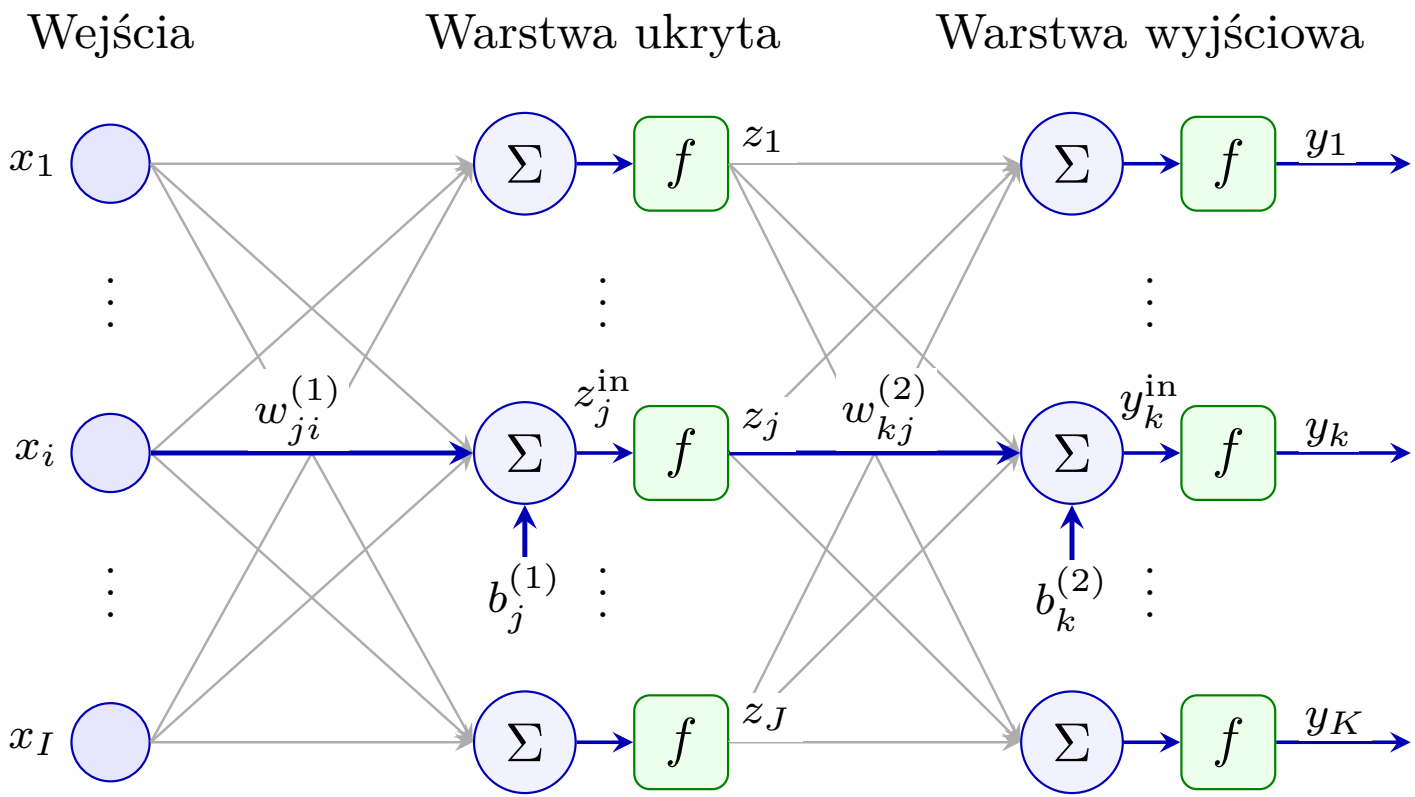
Przejście w przód

1. Sygnały x_i trafiają do neuronów warstwy ukrytej.
2. Neuron ukryty oblicza sumę z_j^{in} i po aktywacji przekazuje sygnał z_j.
3. Neuron wyjściowy oblicza sumę y_k^{in} i po aktywacji generuje odpowiedź y_k.

Przejście wstecz i aktualizacja

4. Porównuje się y_k z t_k a następnie wyznacza wskaźnik korekty δ_k.
5. Z wartości δ_k, wag $w_{kj}^{(2)}$ i aktywacji neuronu ukrytego obliczany jest wskaźnik δ_j.
6. Wskaźniki δ_k i δ_j służą do aktualizacji wag oraz przesunięć obu warstw.

MLP z jedną warstwą ukrytą



$$z_j^{\text{in}} = \sum_{i=1}^{I} w_{ji}^{(1)} x_i + b_j^{(1)}, \quad z_j = f(z_j^{\text{in}}),$$

$$y_k^{\text{in}} = \sum_{j=1}^{J} w_{kj}^{(2)} z_j + b_k^{(2)}, \quad y_k = f(y_k^{\text{in}}).$$

Uczenie MLP – nazewnictwo

Wejście $(i = 1, \ldots, I)$

$\mathbf{x} = [x_1, \ldots, x_i, \ldots, x_I]^T$.

Warstwa ukryta $(j = 1, \ldots, J)$

$w_{ji}^{(1)}$, $b_j^{(1)}$ – waga i przesunięcie;

z_j^{in} – suma przed aktywacją;

$z_j = f(z_j^{\text{in}})$ – sygnał przekazany dalej;

δ_j – wskaźnik korekty neuronu ukrytego.

Warstwa wyjściowa $(k = 1, \ldots, K)$

$w_{kj}^{(2)}$, $b_k^{(2)}$ – waga i przesunięcie;

y_k^{in} – suma przed aktywacją;

$y_k = f(y_k^{\text{in}})$ – odpowiedź neuronu;

δ_k – wskaźnik korekty neuronu wyjściowego.

Oczekiwane i uzyskane odpowiedzi

$\mathbf{t} = [t_1, \ldots, t_k, \ldots, t_K]^T$,

$\mathbf{y} = [y_1, \ldots, y_k, \ldots, y_K]^T$.

Uczenie MLP – pochodne funkcji sigmoidalnych

Pochodną funkcji aktywacji liczymy względem jej argumentu u.

W MLP jest nim z_j^{in} lub y_k^{in}. Przyjmując $\alpha > 0$:

Funkcja unipolarna

$$f_{\text{u}}(u) = \frac{1}{1 + e^{-\alpha u}}, \tag{15}$$

$$f'_{\text{u}}(u) = \frac{\alpha e^{-\alpha u}}{(1 + e^{-\alpha u})^2} = \alpha f_{\text{u}}(u)\big[1 - f_{\text{u}}(u)\big]. \tag{16}$$

Funkcja bipolarna

$$f_{\text{b}}(u) = \frac{2}{1 + e^{-\alpha u}} - 1 = 2 f_{\text{u}}(u) - 1, \tag{17}$$

$$f'_{\text{b}}(u) = 2 f'_{\text{u}}(u) = \frac{\alpha}{2}\big[1 - f_{\text{b}}(u)^2\big]. \tag{18}$$

Uczenie MLP: faza w przód

Przed uczeniem: inicjalizujemy wagi i przesunięcia obu warstw.

1. Neuron ukryty wyznacza sumę przed aktywacją:

$$z_j^{\text{in}} = \sum_{i=1}^{I} w_{ji}^{(1)} x_i + b_j^{(1)}, \qquad j = 1, \ldots, J. \tag{19}$$

2. Stąd sygnał neuronu ukrytego:

$$z_j = f(z_j^{\text{in}}), \qquad j = 1, \ldots, J. \tag{20}$$

3. Neuron wyjściowy wyznacza sumę przed aktywacją:

$$y_k^{\text{in}} = \sum_{j=1}^{J} w_{kj}^{(2)} z_j + b_k^{(2)}, \qquad k = 1, \ldots, K. \tag{21}$$

4. Stąd odpowiedź neuronu wyjściowego:

$$y_k = f(y_k^{\text{in}}), \qquad k = 1, \ldots, K. \tag{22}$$

5. Błąd dla przykładu uczącego $(\mathbf{x}, \mathbf{t})$:

$$E = \frac{1}{2} \sum_{k=1}^{K} (t_k - y_k)^2. \tag{23}$$

Faza wsteczna: warstwa wyjściowa

Dla ustalonych k i j waga $w_{kj}^{(2)}$ wpływa na odpowiedź y_k:

$$\begin{aligned}\frac{\partial E}{\partial w_{kj}^{(2)}} &= \frac{\partial}{\partial w_{kj}^{(2)}}\frac{1}{2}\sum_{n=1}^{K}(t_n - y_n)^2 = \frac{\partial}{\partial w_{kj}^{(2)}}\frac{1}{2}(t_k - y_k)^2 \\ &= -(t_k - y_k)\frac{\partial}{\partial w_{kj}^{(2)}}y_k = -(t_k - y_k)f'(y_k^{\text{in}})\frac{\partial}{\partial w_{kj}^{(2)}}y_k^{\text{in}} \\ &= -(t_k - y_k)f'(y_k^{\text{in}})\frac{\partial}{\partial w_{kj}^{(2)}}\left(\sum_{m=1}^{J} w_{km}^{(2)} z_m + b_k^{(2)}\right) \\ &= -(t_k - y_k)f'(y_k^{\text{in}})z_j.\end{aligned}$$

Definiując wskaźnik korekty neuronu wyjściowego:

$$\delta_k \triangleq (t_k - y_k)f'(y_k^{\text{in}}), \tag{24}$$

aktualizacje wag i przesunięcia wynoszą:

$$\Delta w_{kj}^{(2)} = -\eta\frac{\partial E}{\partial w_{kj}^{(2)}} = \eta\delta_k z_j, \quad \Delta b_k^{(2)} = -\eta\frac{\partial E}{\partial b_k^{(2)}} = \eta\delta_k. \tag{25}$$

Faza wsteczna: warstwa ukryta

Dla ustalonych j i i waga $w_{ji}^{(1)}$ wpływa na wszystkie odpowiedzi y_k.

$$\begin{aligned}\frac{\partial E}{\partial w_{ji}^{(1)}} &= \frac{\partial}{\partial w_{ji}^{(1)}}\frac{1}{2}\sum_{k=1}^{K}(t_k - y_k)^2 = -\sum_{k=1}^{K}(t_k - y_k)\frac{\partial}{\partial w_{ji}^{(1)}}y_k \\ &= -\sum_{k=1}^{K}(t_k - y_k)f'(y_k^{\text{in}})\frac{\partial}{\partial w_{ji}^{(1)}}y_k^{\text{in}} = -\sum_{k=1}^{K}\delta_k\frac{\partial}{\partial w_{ji}^{(1)}}\left(\sum_{m=1}^{J}w_{km}^{(2)}z_m + b_k^{(2)}\right) \\ &= -\sum_{k=1}^{K}\delta_k w_{kj}^{(2)}\frac{\partial}{\partial w_{ji}^{(1)}}z_j = -\sum_{k=1}^{K}\delta_k w_{kj}^{(2)}f'(z_j^{\text{in}})x_i.\end{aligned}$$

Definiując wskaźnik korekty neuronu ukrytego:

$$\delta_j \triangleq \sum_{k=1}^{K}\delta_k w_{kj}^{(2)}f'(z_j^{\text{in}}), \tag{26}$$

aktualizacje wag i przesunięcia wynoszą:

$$\Delta w_{ji}^{(1)} = -\eta\frac{\partial E}{\partial w_{ji}^{(1)}} = \eta\delta_j x_i, \quad \Delta b_j^{(1)} = -\eta\frac{\partial E}{\partial b_j^{(1)}} = \eta\delta_j. \tag{27}$$

Wsteczna propagacja dla sieci trójwarstwowej

Wejścia x_i trafiają do dwóch warstw ukrytych i warstwy wyjściowej. Te trzy warstwy mają funkcje aktywacji, odpowiednio f, g i h.

Sygnał neuronu w pierwszej warstwie ukrytej:

$$z_j = f(z_j^{\text{in}}) = f\left(\sum_{i=1}^{I} w_{ji}^{(1)} x_i + b_j^{(1)}\right), \qquad j = 1, \ldots, J. \tag{28}$$

Sygnał neuronu w drugiej warstwie ukrytej:

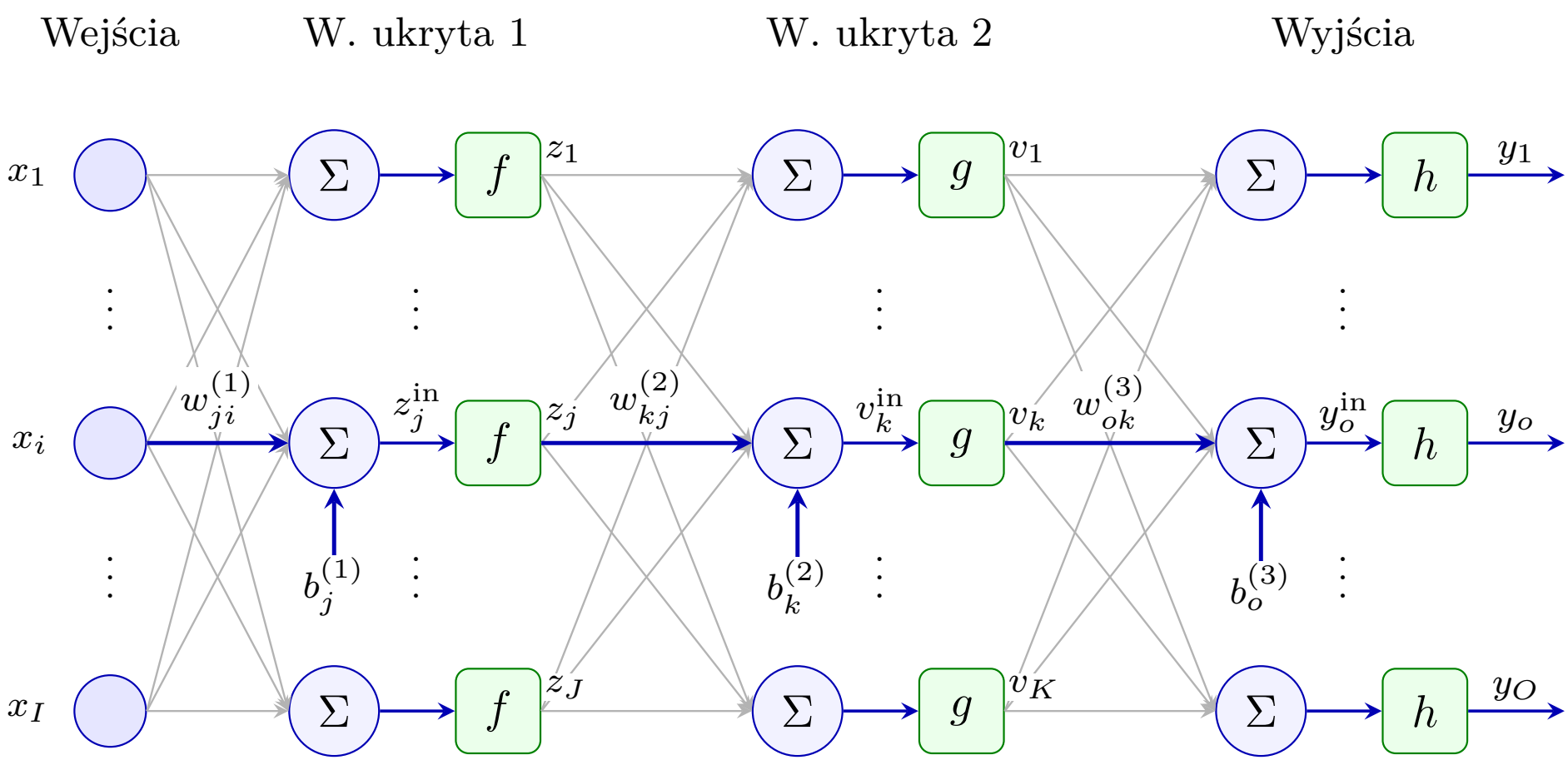
$$v_k = g(v_k^{\text{in}}) = g\left(\sum_{j=1}^{J} w_{kj}^{(2)} z_j + b_k^{(2)}\right), \qquad k = 1, \ldots, K. \tag{29}$$

Odpowiedź neuronu w warstwie wyjściowej:

$$y_o = h(y_o^{\text{in}}) = h\left(\sum_{k=1}^{K} w_{ok}^{(3)} v_k + b_o^{(3)}\right), \qquad o = 1, \ldots, O. \tag{30}$$

Dalej, dla uproszczenia, przyjmiemy $f = g = h$.

Sieć trójwarstwowa – architektura



Σ – suma ważona z przesunięciem; f, g, h – funkcje aktywacji.

Faza wsteczna: warstwa wyjściowa

$$
\begin{aligned}
\frac{\partial E}{\partial w_{ok}^{(3)}} &= \frac{\partial}{\partial w_{ok}^{(3)}} \frac{1}{2} \sum_{p=1}^{O} (t_p - y_p)^2 = \frac{\partial}{\partial w_{ok}^{(3)}} \frac{1}{2} (t_o - y_o)^2 \\
&= -(t_o - y_o) \frac{\partial}{\partial w_{ok}^{(3)}} y_o = -(t_o - y_o) f'(y_o^{\text{in}}) \frac{\partial}{\partial w_{ok}^{(3)}} y_o^{\text{in}} \\
&= -(t_o - y_o) f'(y_o^{\text{in}}) \frac{\partial}{\partial w_{ok}^{(3)}} \left(\sum_{r=1}^{K} w_{or}^{(3)} v_r + b_o^{(3)} \right) \\
&= -(t_o - y_o) f'(y_o^{\text{in}}) v_k.
\end{aligned}
$$

Stąd:

$$
\delta_o \triangleq (t_o - y_o) f'(y_o^{\text{in}}). \tag{31}
$$

Faza wsteczna: warstwa druga

$$\frac{\partial E}{\partial w_{kj}^{(2)}} = \frac{\partial}{\partial w_{kj}^{(2)}} \frac{1}{2} \sum_{o=1}^{O} (t_o - y_o)^2 = -\sum_{o=1}^{O} (t_o - y_o) \frac{\partial}{\partial w_{kj}^{(2)}} y_o$$

$$= -\sum_{o=1}^{O} (t_o - y_o) f'(y_o^{\text{in}}) \frac{\partial}{\partial w_{kj}^{(2)}} y_o^{\text{in}} = -\sum_{o=1}^{O} \delta_o \frac{\partial}{\partial w_{kj}^{(2)}} \left(\sum_{p=1}^{K} w_{op}^{(3)} v_p + b_o^{(3)} \right)$$

$$= -\sum_{o=1}^{O} \delta_o w_{ok}^{(3)} \frac{\partial}{\partial w_{kj}^{(2)}} v_k = -\sum_{o=1}^{O} \delta_o w_{ok}^{(3)} f'(v_k^{\text{in}}) \frac{\partial}{\partial w_{kj}^{(2)}} v_k^{\text{in}}$$

$$= -\sum_{o=1}^{O} \delta_o w_{ok}^{(3)} f'(v_k^{\text{in}}) \frac{\partial}{\partial w_{kj}^{(2)}} \left(\sum_{r=1}^{J} w_{kr}^{(2)} z_r + b_k^{(2)} \right) = -\sum_{o=1}^{O} \delta_o w_{ok}^{(3)} f'(v_k^{\text{in}}) z_j.$$

Stąd:

$$\delta_k \triangleq \sum_{o=1}^{O} \delta_o w_{ok}^{(3)} f'(v_k^{\text{in}}). \tag{32}$$

Faza wsteczna: warstwa pierwsza

$$\frac{\partial E}{\partial w_{ji}^{(1)}} = \frac{\partial}{\partial w_{ji}^{(1)}} \frac{1}{2} \sum_{o=1}^{O} (t_o - y_o)^2 = \ldots = -\sum_{o=1}^{O} \delta_o \frac{\partial}{\partial w_{ji}^{(1)}} \left(\sum_{k=1}^{K} w_{ok}^{(3)} v_k + b_o^{(3)} \right)$$

$$= -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} \frac{\partial}{\partial w_{ji}^{(1)}} \left(w_{ok}^{(3)} v_k + b_o^{(3)} \right) = -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} \frac{\partial}{\partial w_{ji}^{(1)}} v_k$$

$$= -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) \frac{\partial}{\partial w_{ji}^{(1)}} v_k^{\text{in}} = -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) \frac{\partial}{\partial w_{ji}^{(1)}} \left(\sum_{p=1}^{J} w_{kp}^{(2)} z_p + b_k^{(2)} \right)$$

$$= -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) w_{kj}^{(2)} \frac{\partial}{\partial w_{ji}^{(1)}} z_j = -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) w_{kj}^{(2)} f'(z_j^{\text{in}}) \frac{\partial}{\partial w_{ji}^{(1)}} z_j^{\text{in}}$$

$$= -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) w_{kj}^{(2)} f'(z_j^{\text{in}}) \frac{\partial}{\partial w_{ji}^{(1)}} \left(\sum_{r=1}^{I} w_{jr}^{(1)} x_r + b_j^{(1)} \right)$$

$$= -\sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) w_{kj}^{(2)} f'(z_j^{\text{in}}) x_i. \qquad \text{Stąd:} \quad \delta_j \triangleq \sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) w_{kj}^{(2)} f'(z_j^{\text{in}}).$$

Wsteczna propagacja MLP – etapy

❶ **Obliczenie wskaźników korekty** dla aktualnych wag, kolejno od warstwy wyjściowej:

$$\delta_o = (t_o - y_o) f'(y_o^{\text{in}}),$$

$$\delta_k = f'(v_k^{\text{in}}) \sum_{o=1}^{O} \delta_o w_{ok}^{(3)},$$

$$\delta_j = f'(z_j^{\text{in}}) \sum_{k=1}^{K} \delta_k w_{kj}^{(2)} = f'(z_j^{\text{in}}) \sum_{o=1}^{O} \delta_o \sum_{k=1}^{K} w_{ok}^{(3)} f'(v_k^{\text{in}}) w_{kj}^{(2)}.$$

❷ **Aktualizacja wag i przesunięć** po obliczeniu wszystkich wskaźników:

$$\Delta w_{ok}^{(3)} = \eta \delta_o v_k, \qquad \Delta b_o^{(3)} = \eta \delta_o, \tag{33}$$

$$\Delta w_{kj}^{(2)} = \eta \delta_k z_j, \qquad \Delta b_k^{(2)} = \eta \delta_k, \tag{34}$$

$$\Delta w_{ji}^{(1)} = \eta \delta_j x_i, \qquad \Delta b_j^{(1)} = \eta \delta_j. \tag{35}$$

Problem zanikających gradientów

Dla neuronu w warstwie ukrytej:

$$\delta_j = f'(z_j^{\text{in}}) \sum_{k=1}^{K} \delta_k w_{kj}^{(2)}, \qquad \Delta w_{ji}^{(1)} = \eta \delta_j x_i.$$

Gdy sigmoida pracuje w obszarze nasycenia, $f'(z_j^{\text{in}})$ jest bliskie zeru.

Wtedy zmiany wag pierwszej warstwy mogą być bardzo małe.

W sieci z większą liczbą warstw efekt ten może się nasilać.

Co pomaga?

- ReLU w warstwie ukrytej: dla dodatniego argumentu jej pochodna wynosi 1;
- odpowiedni dobór skali wag początkowych.

ReLU ma pochodną równą 0 dla argumentów ujemnych, dlatego nie eliminuje wszystkich trudności związanych z uczeniem.

MLP – ReLU, softmax i entropia krzyżowa

W rozważanej MLP: fun. ReLU w warstwie ukrytej i softmax w warstwie wyjściowej:

- **ReLU** w warstwie ukrytej:

$$f(x) = \max(0, x) = \begin{cases} x & \text{dla } x > 0, \\ 0 & \text{dla } x \leq 0. \end{cases} \tag{36}$$

- **Softmax** przekształca sygnały y_k^{in} całej warstwy wyjściowej w prawdopodobieństwa klas:

$$y_k = \frac{e^{y_k^{\text{in}}}}{\sum_{n=1}^{K} e^{y_n^{\text{in}}}}, \qquad r = 1, \ldots, K. \tag{37}$$

  Wówczas $y_k > 0$ oraz $\sum_{k=1}^{K} y_k = 1$.

Dla pojedynczego przykładu uczącego i wektora $\mathbf{t}$ wskazującego poprawną klasę funkcją straty jest **entropia krzyżowa**:

$$L = -\sum_{k=1}^{K} t_k \ln y_k. \tag{38}$$

MLP-ReLU+softmax: warstwa wyjściowa

Dla ustalonych k i j waga $w_{kj}^{(2)}$ zmienia sygnał y_k^{in}. Ponieważ wyjścia softmaxu mają wspólny mianownik, zmiana tego sygnału wpływa na każde y_r.

$$\begin{aligned}\frac{\partial L}{\partial w_{kj}^{(2)}} &= \frac{\partial}{\partial w_{kj}^{(2)}}\left(-\sum_{r=1}^{K} t_r \ln y_r\right) = -\sum_{r=1}^{K} t_r \frac{1}{y_r}\frac{\partial y_r}{\partial w_{kj}^{(2)}} \\ &= -\sum_{r=1}^{K} t_r \frac{1}{y_r} f'_{rk}\frac{\partial y_k^{\text{in}}}{\partial w_{kj}^{(2)}} = -\sum_{r=1}^{K} t_r \frac{1}{y_r} f'_{rk} z_j.\end{aligned}$$

gdzie: f'_{rk} to pochodna cząstkowa wyjścia y_r względem wejścia y_k^{in} funkcji softmax:

$$f'_{rk} \triangleq \frac{\partial y_r}{\partial y_k^{\text{in}}} = \begin{cases} y_k(1-y_k) & \text{dla } r = k, \\ -y_r y_k & \text{dla } r \neq k. \end{cases} \tag{39}$$

Waga $w_{kj}^{(2)}$ wpływa na sygnał y_k^{in} neuronu k. Softmax łączy wyjścia.

Stąd: $f'_{rk} = \frac{\partial y_r}{\partial y_k^{\text{in}}}$ opisuje wpływ k-tego sygnału na wyjście y_r.

MLP-ReLU+softmax: warstwa wyjściowa ...

$$\begin{aligned}\frac{\partial L}{\partial w_{kj}^{(2)}} &= -\left[t_k \frac{1}{y_k} y_k(1-y_k) + \sum_{r\neq k} t_r \frac{1}{y_r}(-y_r y_k)\right] z_j \\ &= -\left[t_k(1-y_k) - \sum_{r\neq k} t_r y_k\right] z_j = -\left[t_k - t_k y_k - y_k \sum_{r\neq k} t_r\right] z_j.\end{aligned}$$

Dla wektora one-hot $\sum_{r=1}^{K} t_r = 1 \Longrightarrow \sum_{r\neq k} t_r = 1 - t_k$ dlatego:

$$\begin{aligned}\frac{\partial L}{\partial w_{kj}^{(2)}} &= -[t_k - t_k y_k - y_k(1-t_k)]\, z_j \\ &= -[t_k - t_k y_k - y_k + y_k t_k]\, z_j = -[t_k - y_k]\, z_j.\end{aligned}$$

Przyjmując:

$$\delta_k \triangleq (t_k - y_k), \tag{40}$$

aktualizacje wag i przesunięcia są następujące:

$$\Delta w_{kj}^{(2)} = -\eta \frac{\partial L}{\partial w_{kj}^{(2)}} = \eta \delta_k z_j, \quad \Delta b_k^{(2)} = \eta \delta_k \tag{41}$$

MLP-ReLU+softmax: warstwa pierwsza

$$
\begin{aligned}
\frac{\partial L}{\partial w_{ji}^{(1)}} &= \frac{\partial}{\partial w_{ji}^{(1)}}\left(-\sum_{r=1}^{K} t_r \ln y_r\right) = -\sum_{r=1}^{K} t_r \frac{1}{y_r}\frac{\partial y_r}{\partial w_{ji}^{(1)}} \\
&= -\sum_{r=1}^{K} t_r \frac{1}{y_r}\sum_{k=1}^{K} f'_{rk}\frac{\partial y_k^{\text{in}}}{\partial w_{ji}^{(1)}} \\
&= -\sum_{r=1}^{K} t_r \frac{1}{y_r}\sum_{k=1}^{K} f'_{rk}\frac{\partial}{\partial w_{ji}^{(1)}}\left(\sum_{m=1}^{J} w_{km}^{(2)} z_m + b_k^{(2)}\right) \\
&= -\sum_{k=1}^{K}\sum_{r=1}^{K} t_r \frac{1}{y_r} f'_{rk} w_{kj}^{(2)} \frac{\partial z_j}{\partial w_{ji}^{(1)}} = -\sum_{k=1}^{K}\left[\sum_{r=1}^{K} t_r \frac{1}{y_r} f'_{rk}\right] w_{kj}^{(2)} f'(z_j^{\text{in}}) x_i \\
&= -f'(z_j^{\text{in}}) x_i \sum_{k=1}^{K} \delta_k w_{kj}^{(2)}.
\end{aligned}
$$

Ostatni krok wykorzystuje $\sum_{r=1}^{K} t_r \frac{1}{y_r} f'_{rk} = t_k - y_k = \delta_k$.
Pochodna funkcji ReLU:

$$
f'(z) = \begin{cases} 1 & \text{dla } z > 0, \\ 0 & \text{dla } z < 0. \end{cases} \tag{42}
$$

W punkcie $z = 0$ przyjmujemy na potrzeby uczenia $f'(0) = 0$.

MLP-ReLU+softmax: warstwa pierwsza ...

Przyjmując:

$$\delta_j \triangleq f'(z_j^{\text{in}}) \sum_{k=1}^{K} \delta_k w_{kj}^{(2)}, \tag{43}$$

gdzie δ_k zdefiniowano w (40), otrzymujemy aktualizacje wag i przesunięcia:

$$\begin{aligned} \Delta w_{ji}^{(1)} &= -\eta \frac{\partial L}{\partial w_{ji}^{(1)}} = \eta \delta_j x_i, \\ \Delta b_j^{(1)} &= -\eta \frac{\partial L}{\partial b_j^{(1)}} = \eta \delta_j. \end{aligned} \tag{44}$$

Podsumowanie:

- δ_j zależy od wskaźników δ_k neuronów wyjściowych oraz łączących je z neuronem j wag $w_{kj}^{(2)}$;
- δ_j obliczana jest przy użyciu wag $w_{kj}^{(2)}$ sprzed ich aktualizacji.

MLP z sigmoidami: inicjalizacja wag i przesunięć

Metoda Glorota-Bengio (Glorot and Bengio, 2010) dobiera skalę wag na podstawie liczby połączeń w warstwie:

Możliwe są dwa warianty:

- rozkład jednostajny:

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right); \tag{45}$$

- rozkład normalny:

$$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}+n_{\text{out}}}\right), \tag{46}$$

gdzie n_{in} i n_{out} oznaczają liczby neuronów w poprzedniej i bieżącej warstwie: dla $W^{(1)}$ są to I, J, a dla $W^{(2)}$ to J, K.

Przesunięcia można początkowo ustawić na 0.

Inne podejście (Nguyen and Widrow, 1990): dobiera także przesunięcia neuronów ukrytych.

MLP z ReLU: inicjalizacja wag i przesunięć

W MLP z warstwą ukrytą ReLU wagi $w_{ji}^{(1)}$ można zainicjalizować metodą He (He et al., 2015). Tutaj $n_{\text{in}} = I$.

- rozkład normalny:

$$w_{ji}^{(1)} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right); \tag{47}$$

- rozkład jednostajny:

$$w_{ji}^{(1)} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right). \tag{48}$$

Rozkład jednostajny jest wariantem o tej samej wariancji wag.

Wagi warstwy wyjściowej $w_{kj}^{(2)}$ można zainicjalizować metodą Glorota, przyjmując $n_{\text{in}} = J$ i $n_{\text{out}} = K$.

Przesunięcia $b_j^{(1)}$ i $b_k^{(2)}$ można początkowo ustawić na 0.

Inicjalizacja wag – dlaczego metoda He?

ReLU zeruje ujemne sygnały. Jeśli sygnały przed ReLU są początkowo rozłożone symetrycznie wokół zera, około połowa z nich zostaje wyzerowana.

Jak uwzględnić to podczas inicjalizacji?

Przed uczeniem wybieramy rozkład, z którego losujemy każdą wagę $w_{ji}^{(1)}$. Metoda He określa wariancję tego rozkładu:

$$\mathrm{Var}\left(w_{ji}^{(1)}\right) = \frac{2}{n_{\text{in}}}, \qquad n_{\text{in}} = I. \tag{49}$$

Przykładowo, gdy $I = 100$, wariancja wynosi 0.02. Jeśli wybierzemy rozkład normalny, zapisujemy:

$$w_{ji}^{(1)} \sim \mathcal{N}(0,\, 0.02). \tag{50}$$

Wariancja nie jest obliczana z danych uczących ani z pojedynczej wylosowanej wagi.

Określa ona rozrzut możliwych wartości wag na początku uczenia.

Sam proces uczenia

W dotychczasowym wyprowadzeniu wagi i przesunięcia aktualizowane są po jednym przykładzie uczącym $(\mathbf{x}, \mathbf{t})$.

Epoka: jedno przejście przez cały zbiór uczący złożony z L przykładów.

Krok uczenia: jedna aktualizacja wag i przesunięć.

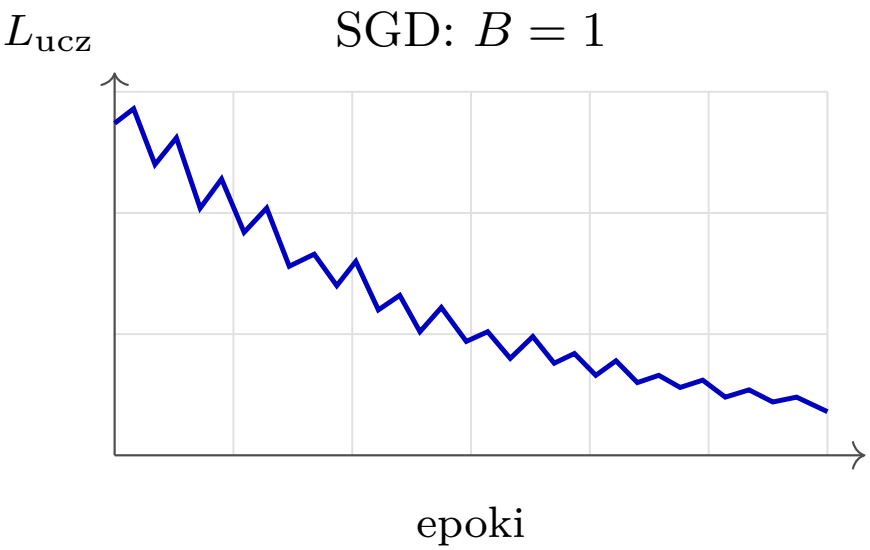
Liczba przykładów użytych do jednego kroku zależy od sposobu uczenia:

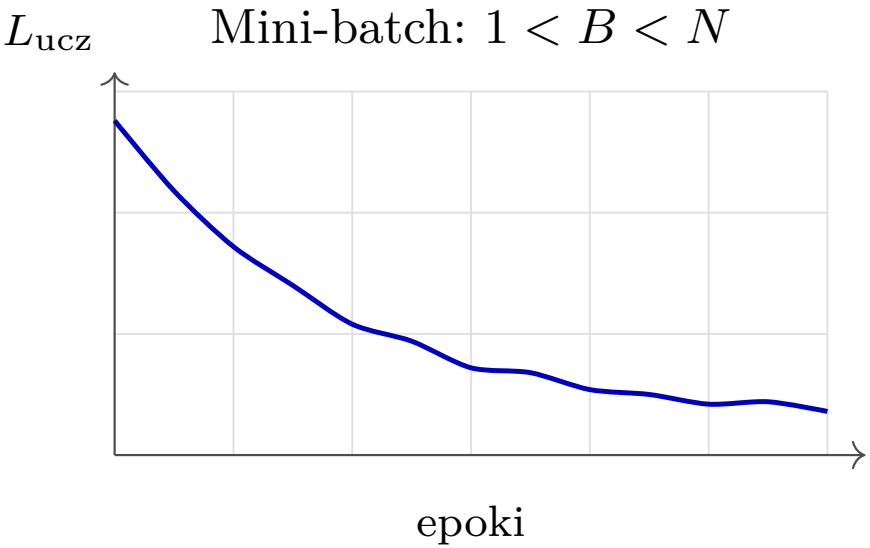
- $B = 1$: jeden przykład na krok (SGD, uczenie po pojedynczym przykładzie);
- $1 < B < L$: porcja B przykładów na krok (*mini-batch*);
- $B = N$: cały zbiór na krok (*full batch*).

Przykład: dla $L = 128$ i $B = \{1, 32, 128\}$ wykonywane są odpowiednio: 128, 4 oraz 1 aktualizację w każdej epoce.

Zbyt krótki trening może prowadzić do niedouczenia, a zbyt długi do przeuczenia; liczbę epok dobieramy na podstawie walidacji.

SGD a mini-batch





SGD $(B = 1)$

- Gradient z jednego przykładu.
- Zmienny kierunek aktualizacji; strata może oscylować.

Mini-batch $(1 < B < L)$

- Gradient z porcji przykładów: zwykle mniejsze wahania.
- Możliwość obliczeń równoległych.

Uwaga: wielkość porcji nie rozstrzyga o przeuczeniu.

Współczynnik uczenia η

Parametr η skaluje krok aktualizacji: $\Delta w = -\eta\, \partial L/\partial w$.

- **Zbyt duże** η: strata może silnie oscylować/rosnąć $\rightarrow$ uczenie niestabilne.
- **Zbyt małe** η: strata maleje bardzo wolno; potrzeba więcej aktualizacji.

☎ **Ważna jest także definicja straty dla porcji B przykładów.**

Jeśli L_m oznacza stratę dla m-tego przykładu w porcji, to

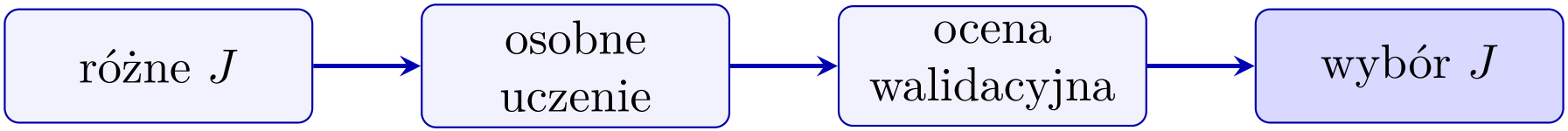
$$L_{\text{suma}} = \sum_{m=1}^{B} L_m, \qquad L_{\text{śr}} = \frac{1}{B} \sum_{m=1}^{B} L_m. \tag{51}$$

Dla tej samej porcji i tej samej wagi w:

$$\frac{\partial L_{\text{suma}}}{\partial w} = B\, \frac{\partial L_{\text{śr}}}{\partial w}. \tag{52}$$

Wniosek: dla tego samego η: suma daje B razy większy krok niż średnia.

Dobór liczby neuronów ukrytych



- Zacznij od jednej warstwy ukrytej i kilku wartości J.
- Większe J oznacza więcej wag.
- Mniejsza strata na zbiorze uczącym nie musi oznaczać lepszego wyniku na walidacji.
- Porównuj sieci przy tym samym podziale danych i sposobie uczenia.
- Zbiór testowy wykorzystaj dopiero po wyborze J.

Twierdzenie o aproksymacji nie wskazuje liczby J dla konkretnych danych (Cybenko, 1989; Hornik et al., 1989).

Dobór architektury na podstawie walidacji: (Goodfellow et al., 2016).

Momentum w uczeniu MLP: usprawnienie

W spadku gradientu nowy krok wyznacza bieżący gradient.

Momentum uwzględnia poprzedni krok (Polyak, 1964):

$$\boldsymbol{\theta}^{(s+1)} = \boldsymbol{\theta}^{(s)} - \eta \nabla_{\boldsymbol{\theta}} L\left(\boldsymbol{\theta}^{(s)}\right) + \mu\left(\boldsymbol{\theta}^{(s)} - \boldsymbol{\theta}^{(s-1)}\right), \tag{53}$$

gdzie:

- $\boldsymbol{\theta}$ obejmuje wagi i przesunięcia,
- s oznacza numer aktualizacji,
- $0 \leq \mu < 1$ określa wpływ poprzedniego kroku.

Jeśli dana waga przez kilka kroków rośnie, momentum może przyspieszyć jej wzrost.

Jeśli dana waga na przemian rośnie i maleje, momentum może zmniejszyć te wahania.

Sposoby usprawnienia uczenia: dobór i zmiana η, mini-batch, optymalizatory adaptacyjne, np. Adam (Kingma and Ba, 2015).

Wybrane metody optymalizacji w uczeniu MLP

Propagacja wsteczna oblicza gradient funkcji straty.

Metoda optymalizacji określa, jak na jego podstawie zmienić wagi i przesunięcia.

- **Metody wykorzystujące gradient:**
  SGD, momentum, Adam oraz gradienty sprzężone.
- **Metody wykorzystujące przybliżenie informacji o pochodnych drugiego rzędu:**
  quasi-Newtona oraz Levenberga–Marquardta (dla odpowiedniej funkcji straty).
- **Metoda Newtona:**
  korzysta z macierzy drugich pochodnych funkcji straty, zwanej Hesjanem.

Literatura I

Zurada J. M. (1992) Introduction to Artificial Neural Systems. West Publishing Company, St. Paul, MN.

Goodfellow I., Bengio Y., Courville A. (2016) Deep Learning. MIT Press, Cambridge, MA.

Azevedo F. A. C., Carvalho L. R. B., Grinberg L. T., Farfel J. M., Ferretti R. E. L., Leite R. E. P., Jacob Filho W., Lent R., Herculano-Houzel S. (2009) Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain. Journal of Comparative Neurology 513(5), 532–541.

Purves D., Augustine G. J., Fitzpatrick D., Katz L. C., LaMantia A.-S., McNamara J. O., Williams S. M. (red.) (2001) Neuroscience. Wyd. 2. Sinauer Associates, Sunderland, MA.

McCulloch W. S., Pitts W. (1943) A logical calculus of the ideas immanent in nervous activity. Bulletin of Mathematical Biophysics 5, 115–133.

Hecht-Nielsen R. (1987) Counterpropagation networks. Applied Optics 26(23), 4979–4984.

Literatura II

Rosenblatt F. (1958) The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, Psychological Review 65, 386–408

Werbos P. (1974) Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences. PhD thesis, Harvard University

Rumelhart D.E., Hinton G.E., Williams R.J. (1986) Learning Internal Representations by Error Propagation. D.E. Rumelhart, J.L. McClelland, PDP research group, eds. Parallel distributed processing: Explorations in the microstructure of cognition, Volume 1: Foundations. MIT Press, 318–362

Hopfield J.J. (1982) Neural networks and physical systems with emergent collective computational abilities. Proceedings of the National Academy of Sciences of the USA 79(8) 2554–2558

Ackley D.H., Hinton G.E., Sejnowski T.J. (1985) A Learning Algorithm for Boltzmann Machine. Cognitive Science 9(1), 147–169

Kohonen T. (1982) Self-Organized Formation of Topologically Correct Feature Maps. Biological Cybernetics 43, 59–69

Literatura III

Hecht-Nielsen R. (1987) Counterpropagation networks. Applied Optics 26, 4979–4984

Broomhead D.S., Lowe D. (1988) Multivariable functional interpolation and adaptive networks. Complex Systems 2, 321–355

Specht D.F. (1990) Probabilistic Neural Networks. Neural Networks 3, 109–118

Hebb D.O. (1949) The Organization of Behavior, a Neuropsychological Theory. John Wiley, New York

McClelland T.L., Rumelhart D.E., PDP Research Group (1986) Parallel Distributed Processing. The MIT Press, Cambridge

Widrow B. (1962) Generalization and Information Storage in Networks of Adaline 'Neurons'. W "Self-Organizing Systems", Eds. M.C. Jovitz, G.T. Jacobi, G. Goldstein. Spartan Books, Washington, s. 435–461

Glorot X., Bengio Y. (2010) Understanding the difficulty of training deep feedforward neural networks. Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, 249–256

Literatura IV

He K., Zhang X., Ren S., Sun J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. Proceedings of the IEEE International Conference on Computer Vision, 1026–1034

Nguyen D., Widrow B. (1990) Improving the Learning Speed of Two–Layer Neural Networks by Choosing Initial Values of the Adaptive Weights. International Joint Conference on Neural Networks, San Diego, III, 21–26

Fausett L. (1994) Fundamentals of Neural Networks. Prentice Hall, New Jersey

Kolmogorov A.N. (1963) On the Representation of Continuous Functions of Many Variables by Superposition of Continuous Functions of One Variable and Addition. Doklady Akademii Nauk SSSR 144, 679–681

Hornik K.M., Stinchcombe (1989) Multilayer Feedforward Networks Are Universal Approximators. Neural Networks 2(5), 359–366

Hecht–Nielsen R. (1987) Kolmogorov's Mapping Neural Network Existence Theorem. IEEE First International Conference on Neural Networks, San Diego, USA

Fletcher R., Powell M.J.D. (1963) A rapidly convergent descent method for minimization. The Computer Journal 6(2), 163–168

Literatura V

Fletcher R., Reeves R. M. (1964) Function minimization by conjugate gradients. The computer journal 7(2), 149–154

Cybenko G. (1989) Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems 2, 303–314.

Hornik K., Stinchcombe M., White H. (1989) Multilayer feedforward networks are universal approximators. Neural Networks 2(5), 359–366.

Hecht-Nielsen R. (1987) Kolmogorov's mapping neural network existence theorem. Proceedings of the IEEE First International Conference on Neural Networks, vol. III, 11–14.

Goodfellow I., Bengio Y., Courville A. (2016) *Deep Learning*. MIT Press, Cambridge, MA, rozdz. 5.3 i 6.4.

Polyak B. T. (1964) Some methods of speeding up the convergence of iteration methods. USSR Computational Mathematics and Mathematical Physics 4(5), 1–17.

Kingma D. P., Ba J. (2015) Adam: A method for stochastic optimization. Proceedings of the 3rd International Conference on Learning Representations (ICLR).